Lab 6: Code and Branch Coverage Analysis

coverage.py and pytest on pet_boarding.py

Field	Details
Name	Sriviswaja M
SRN	PES1UG24AM288
Section	E
Assigned repository	SETAPESU26/L6_L7_49_pet_boarding
Submission repository	github.com/sriviswaja/SE-Lab-AM288 (Lab 6 folder)

1. Objective

To measure how much of the pet_boarding.py module is exercised by its tests, and to write additional pytest test cases that raise line (code) coverage and branch coverage to at least 90%.

Module under test: pet_boarding.py has three functions: boarding_fee() (fee by pet type, days and premium kennel), admission_ok() (admission rules by age, vaccinations and senior status) and checkout_status() (open / late / closed depending on minutes left, stay length and weekend).

2. Line vs Branch Coverage

- **Line coverage** asks whether every line executed at least once.
- **Branch coverage** asks whether every decision (if / elif / else) evaluated to both True and False. A line can run without every outcome of its condition having been tested, so 100% line coverage does not imply 100% branch coverage.

Commands used:

```
coverage run -m pytest <test_file>.py            # line coverage
coverage run --branch -m pytest <test_file>.py   # branch coverage
coverage report -m                                # report with missing lines
```

3. Results Summary

Coverage figures are for pet_boarding.py (37 statements, 26 branches).

Test file	Tests	Line coverage	Branch coverage
test_pet_boarding.py (starter)	2	38%	32%
CodeCoverageTest_AM288_pet_boarding.py	11	100%	98% (1 partial)
BranchCoverageTest_AM288_pet_boarding.py	26	100%	100% (0 partial)

The starter tests left lines 6, 8, 11, 15-23, 28, 30 and 35-45 unexecuted: both error checks, the premium adjustment, all fee-rate logic, two admission rules and the whole of checkout_status().

4. Code Coverage Tests

CodeCoverageTest_AM288_pet_boarding.py adds the minimum set of tests needed to execute every line once:

Test	Input	Lines reached
Unknown pet type	`boarding_fee("hamster", 3, False)`	raise ValueError (line 6)
Negative days	`boarding_fee("cat", -1, False)`	raise ValueError (line 8)
Cat short stay	`boarding_fee("cat", 5, False) = 40`	non-dog rate (else branch)
Premium dog long stay	`boarding_fee("dog", 20, True) = 765`	premium +1 day, dog rate, >10 surcharge, dog add-on
Senior admission	`admission_ok(10, 3, True)`	senior rule (line 28)
Too young	`admission_ok(0.5, 2, False)`	age < 1 rejection (line 30)
Checkout open / late / closed	`(20,F,F) / (40,T,T) / (5,T,F)`	all of checkout_status()

Result: 100% line coverage

```
(venv) (base) viswaja@Viswajas-MacBook-Air Lab 6 %   coverage run -m pytest CodeCoverageTest_AM288_pet_boarding.py
[  coverage report -m
============================= test session
platform darwin -- Python 3.13.13, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/viswaja/Documents/sem 5/LABS/SE Lab/SE-Lab-AM288/Lab 6
plugins: anyio-4.12.1, asyncio-1.4.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 11 items

CodeCoverageTest_AM288_pet_boarding.py ...........

============================== 11 passed i
Name                                  Stmts   Miss  Cover   Missing
-------------------------------------------------------------------
CodeCoverageTest_AM288_pet_boarding.py      26      0   100%
pet_boarding.py                             37      0   100%
-------------------------------------------------------------------
TOTAL                                       63      0   100%
(venv) (base) viswaja@Viswajas-MacBook-Air Lab 6 %
```

Why branch coverage was still 98%: running the same tests with --branch reports one partial branch, 21->23. The inner check if pet_type == "dog" inside the long-stay block was only ever True (the only long stay tested was a dog). Every line had run, but the path where a non-dog stays more than 10 extra days was never taken. This is exactly the gap line coverage cannot see.

5. Branch Coverage Tests

BranchCoverageTest_AM288_pet_boarding.py makes every decision go both ways, and adds boundary tests at each comparison operator, where off-by-one bugs usually hide:

Decision	True case	False case
pet_type not in INCLUDED_DAYS	"hamster"	"cat", "dog", "rabbit"
days < 0	-1	0 (boundary)
is_premium and pet != "rabbit"	premium dog	premium rabbit, non-premium
extra <= 0	dog 1 day; dog 2 days (extra = 0)	dog 5 days
pet_type == "dog" (rate)	dog	cat, rabbit
extra > 10	extra = 11, 17	extra = 10 (boundary)
inner pet_type == "dog"	dog, extra 17	cat, extra 11 (fixes 21->23)
is_senior	senior with 3 / 2 vaccinations	non-senior
pet_age_years < 1	0.5	1 (boundary), 4
vaccinations >= 2	2	1
is_long_stay / is_weekend	both combinations	both combinations

Decision	True case	False case
minutes >= cutoff	20; 15 (exact cutoff)	10, 40, 5
minutes >= cutoff - 15	10, 40	5, 10 on weekend

Result: 100% branch coverage

```
$ coverage run --branch -m pytest BranchCoverageTest_AM288_pet_boarding.py
collected 26 items
BranchCoverageTest_AM288_pet_boarding.py ..........................        [100%]
========================== 26 passed in 0.03s ==========================
$ coverage report -m
Name                                     Stmts   Miss Branch BrPart  Cover   Missing
------------------------------------------------------------------------------------
BranchCoverageTest_AM288_pet_boarding.py      56      0      0      0   100%
pet_boarding.py                               37      0     26      0   100%
```

See **branch_coverage.png** in the repository for the terminal screenshot.

6. Commits

Commit	Contents
Add code coverage tests (100% line coverage)	CodeCoverageTest_AM288_pet_boarding.py, code_coverage.png, .gitignore
Add branch coverage tests (100% branch coverage)	BranchCoverageTest_AM288_pet_boarding.py, branch_coverage.png

7. Observations

- **Line coverage can be misleading.** The code coverage suite reached 100% of lines with only 11 tests, yet still missed a real path through boarding_fee(). Branch coverage exposed it immediately.
- **Boundary values matter.** Tests at days = 0, extra = 10, age = 1 and minutes = cutoff check the exact edge of each comparison. A change such as > 10 to >= 10 would pass the line-coverage suite but fail these tests.
- **The Missing column guides test writing.** Reading line numbers and branch arcs (like 21->23) from coverage report -m pointed directly to the input that was needed.
- **100% coverage is not proof of correctness.** Coverage shows that code ran, not that the asserted values are right; the tests also assert expected fees and statuses calculated by hand.

8. Conclusion

Starting from 38% line and 32% branch coverage, two test suites were written for pet_boarding.py. The code coverage suite reached 100% line coverage, and the branch coverage suite reached 100% line and 100% branch coverage with all 26 tests passing, exceeding the 90% target for both.